

FSB - FPT University

DDM501 - AI in DevOps, DataOps, MLOps

FINAL PROJECT REPORT

Face Voice Proctoring

Face and Voice Integrity Service

Identity verification and biometric policy lifecycle management
for corporate language assessments

Project team

Trịnh Đức Dương

Đỗ Quang Hiệp

Tô Thanh Hải

Ngô Anh Đức

Report date: 5 October 2026 | English edition

Repository

<https://github.com/FSB-MSA36HN/DDM501-face-voice-proctoring>

Scope: implementation in the current checkout, including local updates. CI and simulation evidence is attributed to its date, revision and run. Synthetic results do not replace a benchmark on real employees.

Contents

1 Executive Summary	3
2 Business Problem and Requirements	3
3 Objectives and Success Criteria	4
4 System Architecture and Design Decisions	5
5 Data and Data Lake Governance	6
6 Models and Evaluation Methods	8
7 Serving and Product Workflow	9
8 Airflow and the Training Pipeline	10
9 Drift Monitoring and the Decision Engine	11
10 Controlled Template Updates	12
11 MLflow and Stateful Policy Deployment	13
12 Monitoring and Alerting	15
13 CI/CD and Application Deployment	16
14 Testing and Verification Results	16
15 Simulation and Demonstrated Results	17
16 Responsible AI and Data Protection	19
17 Operations, Recovery and Scale	20
18 Team Responsibilities and Handover	22
19 Limitations, Roadmap and Conclusions	22
Appendix A Evidence Sources	23
Appendix B Glossary	25

Tables and figures describe the current implementation. Verification results in Chapters 14-15 belong to saved runs from 4 October 2026. Sources and evidence boundaries are listed in Appendix A.

1 Executive Summary

This project delivers face and voice verification for batches of images and WAV recordings submitted during corporate language assessments. The customer retains responsibility for accounts, assessments, capture schedules, scores and business decisions. The service provides APIs, webhooks and portals to verify identity, return suspicious signals, retain appropriate evidence and support authorized human review.

The main outcome is an operational workflow connecting data, inference, monitoring and controlled policy updates. Face and Voice use pretrained encoders; retraining recalibrates decision thresholds rather than updating SFace or ECAPA weights. Each modality has its own reference data, drift decision, MLflow versions and rollout state. A Voice change does not automatically require a Face update.

The pipeline creates a candidate from a versioned snapshot, evaluates it against the champion on the same holdout, and requires shadow and canary stages before changing the champion. Monitoring distinguishes capture quality, embeddings, similarity scores, labeled performance and template age. Statistical drift alone is insufficient to trigger retraining. When evidence is missing, the system reports insufficient data instead of assuming that the model is healthy.

The simulation demonstrates two outcomes: a policy passes all gates and becomes champion, or a policy is stopped during canary because its false match rate increases. The scenarios use synthetic vectors and labels while exercising decision logic, calibration, HTTP routing, alert receipt and MLflow in an isolated environment. They demonstrate MLOps mechanisms, not accuracy on real employees.

Area	Implemented outcome	Important limitation
Product	Batch API, tenant portal, enrollment, history, evidence and webhooks	Does not grade assessments or automatically determine cheating
ML lifecycle	Two independent threshold policies, offline evaluation, shadow, canary and rollback	No encoder fine-tuning
Operations	Airflow, MLflow, MinIO, Prometheus, Grafana and CI/CD	Docker Compose on one host
Verification	CI evidence tied to a SHA, tests and four saved simulation runs from 4 October	Does not establish production accuracy or capacity

Sources: README, ARCHITECTURE, MODALITY_LIFECYCLE and EVIDENCE [S1-S4].

2 Business Problem and Requirements

2.1 Why the project began

Consider an employee signing in with the correct account at 08:55 to take a language assessment. During the speaking section, another person with stronger language skills assists or takes over. The assessment platform still records the submission under the original account. If the result informs training or work assignments, an identity error can distort personnel decisions. This is a hypothetical scenario used to define requirements, not a measured incident at a customer.

Three problems must be addressed: verifying who provided a capture, reducing the effort needed to retrieve evidence, and limiting false accusations. Poor lighting, a weak microphone or an old enrollment template can reduce similarity even when the employee is acting honestly. A binary match result is therefore insufficient. The

system also needs policy versions, capture quality, reason codes and information about capabilities that could not be evaluated.

2.2 Users and responsibility boundaries

Company administrators manage employees, enrollment, integration access and their own tenant's history. The customer's backend authenticates employees, decides when to submit images/WAV files and handles business outcomes. Platform operators manage tenants, the model lifecycle, monitoring and infrastructure. The employee demo page illustrates customer-side integration; its name selector is not a production employee authentication system.

Requirement category	Specific requirement	Verification approach
Functional	Face/Voice matching, integrity signals, history and callbacks	Request, response and canonical check remain consistent
Data	Tenant isolation, idempotency and snapshot lineage	Cross-tenant access is blocked; retries do not create new results
Model	Same-holdout comparison, no immediate promotion and audited rollback	Inspect state, aliases and response versions
Operational	Health, readiness, freshness, logs and alerts	Observe both services and model evidence
Safety	Independent labels, consent and limited raw retention	Predictions are never used as ground truth

The scope does not include continuous video surveillance, detection of every form of coaching or deepfake, or disciplinary decisions. Demo registration/subscription does not implement payments or legal-entity verification [S1, S5].

3 Objectives and Success Criteria

The business objective is timely evidence for human review, not maximizing the number of flagged employees. Technical objectives are to reproduce the data and model behind a decision, maintain observability and control update risk. Model quality must be assessed with independent labels, separately from HTTP availability.

Objective level	Metric or criterion	Current status
Business	Review time, confirmed suspicious cases and errors affecting real people	Not measured at a customer
API	Canonical responses, tenant isolation, idempotency and signed delivery	Implementation and tests exist
Policy quality	FMR, FNMR, EER and TAR at a configured FAR	Methods exist; representative production benchmark is missing
Rollout	Sufficient samples, duration and labels; no FMR regression; rollback	State machine and simulation exist

Objective level	Metric or criterion	Current status
CI	At least 80% coverage within the declared scope; Windows preflight executes	Verify artifacts from the specific run
Scale	Approximately 50,000 managed employees	Design target, not proven by load testing

A pilot could measure review time saved per 1,000 assessments, handling time per suspicious case, recapture rate and cost per check. Business targets in SCALABILITY_COST are hypotheses to validate, not achieved ROI. The current gate limits of 1% FMR and 5% FNMR are pilot configuration values, not statistical guarantees for an entire population.

Without enough impostor labels, a low FMR cannot be inferred from many accepted genuine samples. Likewise, an error-free small demonstration does not establish capacity for 50,000 employees or fairness across groups [S3, S6].

4 System Architecture and Design Decisions

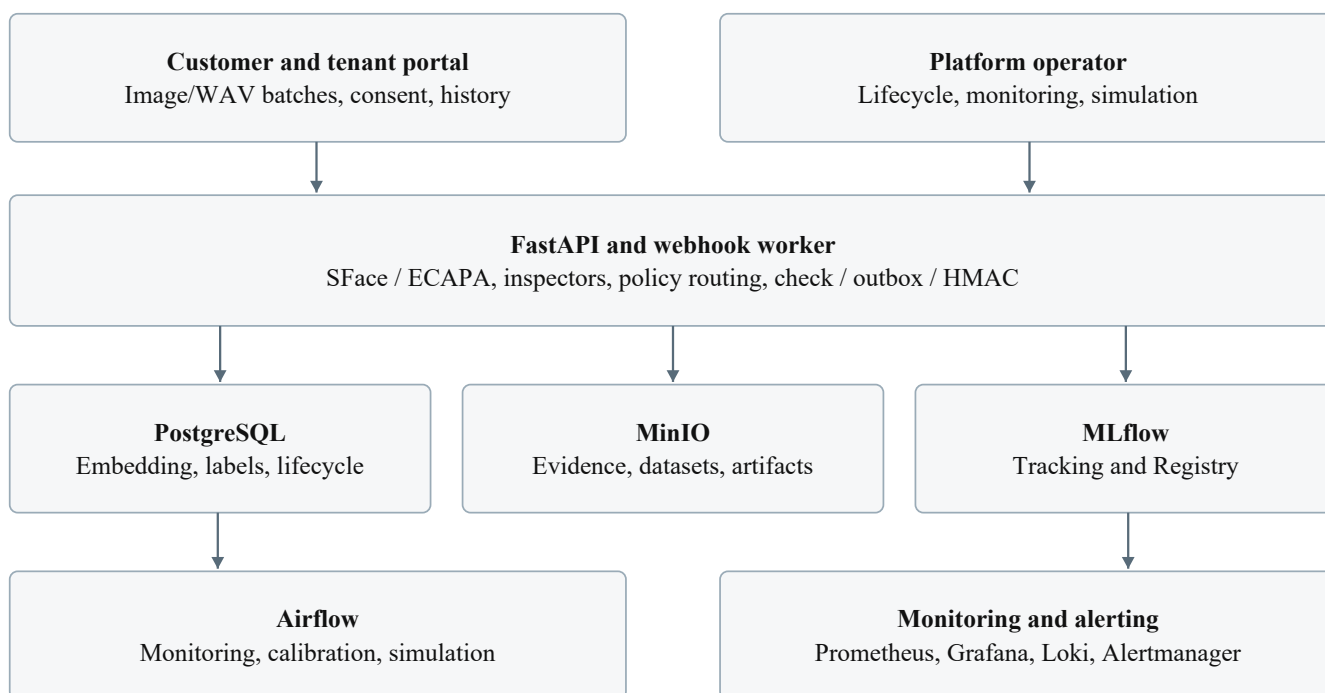


Figure 1. Overview of information flow. Components share controlled data; simulation uses its own database and registry.

The system runs on one machine using Docker Compose. FastAPI contains Face/Voice processing and integrity inspectors as capabilities within the same serving container, rather than a separate server for every model. PostgreSQL stores application data and lifecycle state. MLflow is the registry; the database coordinates routing, state and recoverable transitions.

Component	Responsibility	Main output
FastAPI and Streamlit	Verification, company/platform administration and review	Checks, versions, evidence and history
PostgreSQL	Tenants, JSON embeddings, templates, labels, outbox and lifecycle	Relational data and durable audit records

Component	Responsibility	Main output
MinIO	Object storage for evidence, datasets and artifacts	Objects identified by scope/fingerprint
Airflow	Orchestrates monitoring, calibration and simulation	DAG runs, task states and training intents
MLflow	Experiment tracking and Model Registry	Parameters, metrics, artifacts and aliases
Prometheus and Grafana	Collect metrics and present telemetry	Dashboards, queries and alert evidence
Alertmanager, ops-monitor and Loki	Alert routing, collectors and logs	Operational notifications and investigation data

PostgreSQL JSON embeddings suit 1:1 verification because the person ID is already known. The service does not need nearest-neighbor search across all employees. A vector database would be relevant if query requirements changed or benchmarks justified it; it is not an existing component.

Airflow is separate from request serving so inference does not depend on an active training task. The registry and artifact store separate model identity from the code image. This adds dependencies compared with a standalone application and requires health checks, readiness, backups and consistency checks between the database and registry. CI/CD updates code and containers, while the model lifecycle updates threshold policies inside the API [S2, S7].

5 Data and Data Lake Governance

5.1 Sources and interpretation

Enrollment requires distinct captures of the same person, with at least two accepted images and two accepted WAV recordings for a ready profile. Raw enrollment captures are not retained by default; embeddings and metadata are stored. New queries are used for verification, and reviewed labels are stored separately from predictions to preserve the original decision history.

Technical bootstrap uses LFW from marcelohaps/lfw and Speech Commands from mteb/speech-commands-mini, with revisions pinned in pipeline/bootstrap_demo.py. The two sources are paired into synthetic identities. The face and voice are not confirmed to belong to the same real person. Dataset speaker/identity labels support technical comparison pairs but do not make this a corporate Face+Voice benchmark. Dataset usage rights must be reviewed before redistribution or use beyond the demo.

A saved local validation report at 22:37 on 4 October 2026, Vietnam time, recorded the following snapshot for tenant demo. These are snapshot counts, not permanent totals for the current database.

Modality	Embedding rows	Identities	Dimensions
Face	128	52	128
Voice	154	54	192
Total rows	282	Do not add identities across modalities	Encoder-dependent

The report records `valid=true` and no validation errors. The dataset fingerprint begins `e986b94b66d186b4`. Validation establishes conformance to the code's structural checks, not population representativeness. Source: `reports/data-quality.json` [S8].

5.2 Storage and lineage

Store	Data	Purpose
Application PostgreSQL	People, JSON embeddings/templates, checks, labels, outbox and rollout audits	1:1 serving, tenant scope and traceability
Metadata PostgreSQL	Airflow and MLflow metadata in their respective databases	Run, task, experiment and registry state
MinIO biometric-samples	Suspicious media, training snapshots/manifests, monitoring scalar inputs, labels and reports	Evidence and versioned Data Lake
MinIO MLflow artifacts	Policy JSON, MLmodel, serialized policies, dependencies and evaluation artifacts	Reloading and inspecting model policies
Runtime models	Pinned SFace, ECAPA and detector weights	Pretrained inference
Simulation volume	SQLite, synthetic datasets, file-backed MLflow and run reports	Isolated demonstration

Snapshots are fingerprinted with SHA-256, validated and published before calibration. Model artifacts link the dataset/version, modality and training intent. Monitoring exports scalar inputs, labels and reports to MinIO by tenant, modality and policy version; query vectors are not copied into these monitoring exports. Approved training snapshots may contain embeddings and require a separate retention policy.

`RETAIN_MONITORING_EMBEDDINGS=false` is the default. If retention is authorized and enabled, lifecycle maintenance defaults to 30 days for query vectors and 90 days for observation metadata. This does not automatically delete enrollment records, active templates, all MinIO objects or backups. Suspicious evidence is an exception to ordinary raw-capture disposal. Subscription deactivation blocks access but does not mean data erasure [S2, S3, S8].

6 Models and Evaluation Methods

6.1 Inputs and outputs by layer

Layer	Input	Processing and output
Face encoder	Image containing a face	YuNet detection/alignment; SFace produces a normalized embedding
Voice encoder	WAV recording	Audio validation and preprocessing; ECAPA-TDNN produces a normalized embedding
Matching	Query embedding and the person's active templates	Maximum cosine similarity for each modality
Threshold policy	Similarity and the routed threshold	Face/Voice identity decisions and margins
Integrity checks	Media, metadata and request context	PAD, AASIST, face count, reuse and speaker-change signals
API contract	Combined results	verified, suspicious or inconclusive, with reasons and versions

For normalized vectors, similarity is their dot product. If person *i* has multiple templates, the score is the maximum cosine similarity between the query and that person's templates. The threshold policy reports an identity match when the score reaches the threshold. Capture quality and integrity signals also affect the product result; an identity match alone does not establish that a capture is live or valid.

SFace and ECAPA-TDNN are pretrained rather than trained from scratch by the team. YuNet, MiniFASNet and AASIST also use attributed models/research code with pinned weights. ECAPA is additionally applied to segments for a speaker-change heuristic. This is not full diarization or evidence of detecting every physical replay [S9].

6.2 Calibration and train/validation/test separation

`identity_evaluation` creates five identity partitions with seed 501. One partition, approximately 20% of identities, is reserved as holdout. The remaining four support calibration and four-fold cross-validation. Each CV pass uses approximately 60% of all identities to select a threshold and 20% for validation, while the 20% holdout remains outside tuning. Final fitting uses all 80% calibration identities, followed by evaluation on the 20% holdout.

These proportions refer to identities; row proportions can differ because people have different sample counts. Genuine scoring uses one capture as a probe and other captures of the same identity as templates. Impostor scoring compares the probe with another identity's templates. Maximum-template scoring matches serving behavior, but comparison pairs remain subject to source and sampling bias. Evaluation caps templates per identity and impostor identity pairs to control cost.

The threshold search covers 1,151 values from -0.2 to 0.95. The current minimax objective minimizes the larger of FAR and FRR, using their sum as a secondary criterion. CV selects the smallest margin meeting its budget from 0, 0.002, 0.005, 0.01 and 0.02. Holdout data is excluded from margin selection. The internal calibration/CV budget of 20% is distinct from the lifecycle promotion gates of 1% and 5% [S10].

6.3 Metrics and interpretation

FMR = false accepts / impostor attempts. FNMR = false rejects / genuine attempts. Technical reports use FAR/FRR for the corresponding identity-path error rates. These metrics do not directly measure failure to detect every form of cheating.

EER estimates the crossing point of the two error rates while sweeping thresholds. TAR at a configured FAR is the best genuine acceptance rate satisfying the FAR budget on the evaluation samples. For threshold-only policies sharing the same scores, EER and TAR@FAR may remain identical while FMR/FNMR at the serving threshold differ. Both score separability and the operating threshold must be considered rather than selecting one favorable metric.

7 Serving and Product Workflow

The customer backend submits POST /v1/checks with a person ID, session ID, request ID, consent, images and WAV audio, authenticated by an integration key. Person binding must originate from valid customer-side authentication. A tenant API key must not be exposed in an employee's browser.

Processing validates the tenant and payload, checks idempotency, reads active templates, runs encoders/inspectors, selects Face/Voice policies, and creates the result and check/event/outbox records. The API immediately returns the canonical result. The webhook worker delivers that same result to an allowlisted callback, signs it using HMAC and a timestamp, and retries independently of the original request. Receivers must verify signatures, prevent replay and deduplicate delivery IDs because delivery is at-least-once.

Situation	Required behavior
Same request ID and payload	Return the existing result without creating another check
Same request ID with a different payload	Return conflict 409
Access to another tenant's check/evidence	Block access, generally returning 404 under the contract
Identity mismatch or suspicious integrity signal	Return suspicious with reasons; do not make an accusation
Missing detector or insufficient evaluation data	Return inconclusive/capability unavailable according to the aggregate result
Webhook delivery fails	Preserve the check and outbox for retry
Evidence storage fails	Explicitly report partial/unavailable evidence

Check metadata and the outbox share a PostgreSQL transaction, but MinIO object writes are outside that ACID transaction. If the database fails after upload, an orphan object may remain; production development needs reconciliation and retention controls. The tenant portal reads authorized history/evidence, filters by session, person and time, and exports CSV/PDF.

/health establishes service liveness; /ready checks model-serving prerequisites. An installation with an empty registry may return health 200 and ready 503. Readiness is not a biometric quality evaluation, and an HTTP 200 response does not establish that every capability passed [S5, S7].

8 Airflow and the Training Pipeline

Airflow orchestrates steps rather than acting as the training algorithm. MLflow tracks experiments and registry entries rather than independently deciding when to retrain. Python pipeline code performs snapshotting, validation, calibration and evaluation. The three DAGs have distinct responsibilities.

DAG	Schedule	Responsibility
biometric_monitoring_pipeline	Hourly	Bootstrap a valid incumbent, assess drift, tick rollout and issue eligible training intents
biometric_model_pipeline	schedule=None	Calibrate the threshold for the requested Face or Voice modality
biometric_simulation	Polls every minute	Receive demo jobs and execute drift, alert, training, offline, shadow and canary steps

The model DAG receives modality and window ID in DagRun configuration. It has the following seven tasks and no unconditional weekly training schedule.

Step	Task ID	Actual behavior
1	ingest_versioned_snapshot	Freeze the snapshot and provenance
2	validate_data_quality	Validate vectors, dimensions, samples, identities and data validity
3	publish_versioned_dataset	Store the dataset and manifest in MinIO
4	feature_engineer_train_register_candidate	Calibrate one modality and register an MLflow candidate
5	generate_responsible_ai_audit	Generate an RAI report from available evidence
6	evaluate_and_promote_candidate	Apply offline gates; on success, start challenger/shadow
7	reload_current_champion	Tick the lifecycle; this legacy name does not imply an unconditional champion change

Monitoring does not train inside the drift-calculation code. It creates a deterministic intent for a modality/window, and the lifecycle service uses row locks and state to prevent duplicate jobs. Retrying the same intent does not create an independent training sequence. Training failure is recorded as rejected through a callback. Face and Voice claims are independent, although they share Airflow and host resources.

Training currently supports only tenant demo; data_snapshot.py rejects other tenants. Changing an environment variable is insufficient to train on customer data: data scope, consent enforcement and isolation tests are still required. Lifecycle bootstrap migrates an already registered incumbent bundle into two policies; it does not create an evaluated champion from local-default [S3, S11].

9 Drift Monitoring and the Decision Engine

9.1 Data windows

The lifecycle compares reference and current data separately for each tenant, modality and policy version. The default is 100 observations per window. The reference is frozen and current observations come afterward. Encoders must be compatible and identity-set overlap must reach 80%. Quality, embedding and score comparisons use common identities to reduce changes caused merely by population composition. This does not eliminate all sampling bias.

If the database contains only 100 valid observations and no reference, it cannot yet form two 100-observation windows. With an independent reference of 100 earlier samples, 100 new samples may support one comparison, provided vectors, quality data and labels are appropriate. Refreshing the same window three times does not create three new windows.

9.2 Five evidence layers

Layer	Feature or input	Method and output
Quality	Face brightness, contrast, blur, resolution and detector confidence/area when available; Voice duration, RMS, clipping, silence and sample rate	PSI per scalar; quality score is the largest valid PSI
Embedding	Normalized vectors from the same encoder	Squared RBF MMD, capped at 512 vectors per window
Verification score	Genuine and impostor similarity with trusted labels	Separate PSI, mean/std/quantiles, margin and separation
Performance	Independent labels and recorded scores/thresholds	FMR, FNMR, EER and TAR@FAR; insufficient labels are reported explicitly
Template age	Age of the active template or original enrollment	Sample counts, genuine scores and error rates by age bucket

PSI is calculated as $\text{PSI} = \sum_i [(c_i - r_i) \ln(c_i / r_i)]$, where r_i and c_i are the reference and current proportions in bin i . The implementation uses 10 bins based on reference quantiles and clips probabilities at 10^{-6} to avoid $\log(0)$. A value greater than 0.2 indicates drift under the configuration. Unmeasured features such as SNR, yaw/pitch/roll or occlusion must not be invented in reports. Silence ratio is amplitude-based, not neural VAD.

Embedding drift uses $\text{MMD}^2 = \text{mean}(\mathbf{K}_{rr}) + \text{mean}(\mathbf{K}_{cc}) - 2 \text{mean}(\mathbf{K}_{rc})$, with $\mathbf{K}(\mathbf{x}, \mathbf{y}) = \exp(-\|\mathbf{x} - \mathbf{y}\|^2 / 2)$ on normalized vectors. This is a biased squared MMD estimator including diagonal terms, with threshold 0.02. It is neither a per-dimension test nor a p-value. The interface label MMD refers to this squared formulation.

Score drift separates genuine and impostor observations using trusted labels. A leftward genuine shift or rightward impostor shift can reduce separation. Genuine margin equals similarity minus the verification threshold; a small margin means the score is close to the rejection boundary. Score drift alone, however, does not establish degraded verification performance.

Performance requires at least 20 genuine and 20 impostor labels in each window. Degradation is recorded when FMR or FNMR increases by at least 0.02 in absolute terms and at least 25% relative to baseline. A zero baseline uses the absolute condition. Two percentage points are different from a 2% relative increase. Label sources must

be independent of predictions, and the lifecycle uses a server-selected random-audit cohort to reduce selection bias.

Template age buckets are ≤ 30 , 31-90, 91-180, 181-365 and > 365 days. The current aging heuristic compares the first and last groups: recent FNMR $\leq 5\%$, oldest FNMR $> 7\%$, sufficient performance evidence and overall FMR $\leq 1\%$. This is not a causal correlation model; intermediate groups may still have problems. Cross-age degradation requires at least two buckets to exceed FMR/FNMR budgets [S3, S12].

9.3 Decisions and persistence

Evidence	State	Action
Quality drift	INPUT_DRIFT	Investigate capture conditions; do not automatically retrain
Missing labels/vectors or incompatible cohorts	INSUFFICIENT_DATA	Collect evidence; do not assume healthy performance
Aging heuristic and degraded performance	TEMPLATE_UPDATE_REQUIRED	Wait for persistence and template evidence gates
Persistent embedding and score drift with cross-age performance degradation	RETRAIN_REQUIRED	Check cooldown, new data and idle lifecycle before issuing an intent
Score or embedding drift alone	SCORE_DRIFT or EMBEDDING_DRIFT	Continue monitoring
Degraded performance without a sufficiently established cause	MONITOR	Investigate before updating a model
Measured evidence within limits	HEALTHY	Continue monitoring

Each modality maintains its own persistence. A window counts as new only after at least one full window of new observation IDs has arrived since the previous count. Retraining requires three new eligible windows, at least 40 newly reviewed observations, a 24-hour cooldown, sufficient identities and no active training/rollout. Input and template issues take priority over model updates; statistical drift does not directly invoke training.

The structured output includes modality, model version, windows, per-feature quality PSI, MMD^2 , score summaries, performance status, age buckets, persistence, decision and reasons. Reports and audit records explain why an action occurred or was blocked. With vector retention disabled, automation may stop at insufficient evidence even when score graphs contain data.

10 Controlled Template Updates

A person's template is distinct from a modality's threshold policy. Template updates refresh one person's reference samples; calibration changes a model policy's decision boundary. Changes in one employee's appearance or voice do not automatically require retraining an encoder for the whole system.

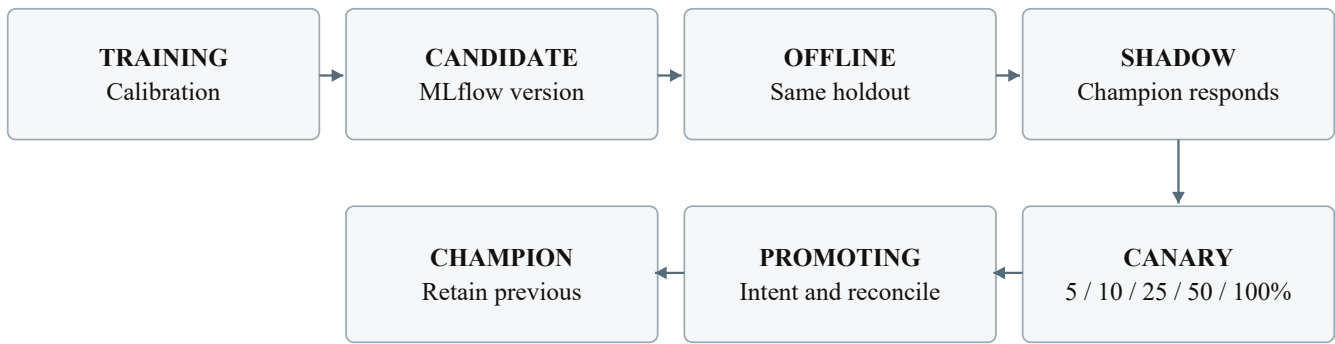
When aging evidence exists, the workflow does not immediately replace a template with the latest accepted capture. Candidates need multiple observations with trusted modality labels, distinct media hashes, valid capture

integrity, high quality, sufficient score margin and no identity conflict. Defaults require at least 10 creation observations, quality ≥ 0.7 , margin ≥ 0.1 and consistency ≥ 0.85 .

A candidate adds a normalized centroid of the observations to the existing template set, keeps original enrollment and caps the set at eight templates per person/modality. Holdout excludes creation hashes and requires at least 20 genuine and 20 impostor labels. FMR must not increase or exceed 1%, and FNMR must not worsen. Insufficient confidence leaves the update in PENDING_REVIEW.

Activation waits for an idle model lifecycle and evidence compatible with the champion, locks the person row, writes a TemplateVersion and switches the ActiveTemplate pointer. The previous version remains available for operator rollback. Authorized new enrollment also updates the relevant template version so serving does not ignore newly enrolled samples. Automatic post-activation monitoring and rollback for templates are not implemented; template rollback is a separate platform action [S3, S13].

11 MLflow and Stateful Policy Deployment



*Gate failure: stop challenger; retain incumbent and failure audit.
 Offline success assigns challenger; only final canary success changes champion.
 Figure 2. One modality lifecycle. Face and Voice have separate states and aliases.*

11.1 Registry and the meaning of champion

Production MLflow manages two registered models: face-verification and voice-verification. The candidate alias identifies a newly registered version; challenger identifies a version that passed offline gates and is under controlled evaluation; champion is the accepted serving policy; previous_champion retains the prior version. These are roles within one lifecycle, not simply aliases for development, test and production environments.

face-voice-risk-bundle is a legacy policy bundle and migration source for the incumbent, not a template or encoder. After initialization of independent lifecycles, routing uses the database's ModalityDeployment state. MLflow stores model identity and artifacts; the database stores traffic, stage time, revision and audit. Manually changing an alias alone does not ensure that serving routing changes correctly.

Candidate and champion are evaluated on the same holdout, including class counts, FMR/FNMR and supporting metrics. A candidate that lowers FNMR while increasing FMR is blocked by the security regression gate. Passing configured criteria does not prove that the candidate is best for every future dataset or strictly improves every metric.

11.2 Shadow and canary

Shadow computes both policies' decisions from the same similarity and records versions, thresholds, latency and disagreement, while real responses still follow the champion. Both policies share the encoder, so score delta is zero; changing the threshold can still cause decision disagreement. Policy p95 latency is not end-to-end latency for upload, encoders and inspectors.

Canary uses a stable hash of modality, deployment, tenant, person and session to select the cohort receiving challenger decisions. Default stages are 5%, 10%, 25%, 50% and 100%. Small-sample observed proportions need not exactly equal configured percentages. Production canary gates use candidate-served observations after the current stage's start time; late labels are considered on lifecycle ticks.

Requirement per shadow or canary stage	Production/pilot default
Minimum observations and elapsed time	1,000 and 3,600 seconds
Minimum trusted labels per class	30
Maximum FMR / FNMR	0.01 / 0.05
Allowed FMR regression	0
Allowed FNMR regression	0.005
Maximum policy p95 / disagreement	10 ms / 0.1
Cohort quality sample threshold	20

Missing data means waiting, not passing because no error has yet been observed. Gates also evaluate cohorts to reduce the risk of pooled metrics hiding deterioration in one group. Active rollout is represented by persistent state rather than inferred from log text [S3, S14].

11.3 Promotion and rollback

Canary failure sets challenger traffic to zero, keeps the champion and records the metric, observed value, limit, timestamp, stage and model version. Failed models remain in MLflow for audit. Polling a rejected candidate again does not automatically retry promotion.

Only a successful final stage creates a durable PROMOTING intent, reconciles registry aliases and then commits the new database champion. If an external call is interrupted, retry resumes the intent; serving keeps the incumbent until the database switch completes. This is a recoverable transition, not a cross-system ACID transaction between PostgreSQL and MLflow.

Rollback during rollout retains the incumbent; rollback after promotion can restore the previous version. Label-dependent gates run on hourly ticks and do not guarantee immediate action when a label is entered. System failures or invalid policies can stop rollout earlier, but a database outage prevents immediate durable state writes. In-flight responses are not recalled. Application rollback is separate from policy rollback.

12 Monitoring and Alerting

Prometheus scrapes API, webhook-worker, drift-monitor, ops-monitor and simulation metrics every 15 seconds. API/worker metrics cover requests, decisions, latency and delivery. ops-monitor collects PostgreSQL, MLflow, Airflow, health-probe and read-only Docker-proxy information and exports metrics. Alloy forwards logs to Loki. Grafana queries telemetry; it does not perform calibration or make drift decisions.

Both current dashboards have 10 panels backed by Prometheus. System Overview focuses on readiness, active incidents, traffic, 5xx errors, verification p95, CPU/RAM, outbox, modality drift, human FAR/FRR and pipeline/lifecycle freshness. Simulation shows synthetic baseline/drift, persistence, alert receipt, offline evaluation, routed samples, candidate errors, latency, outcomes and reset. PostgreSQL, Loki and Alertmanager are also provisioned for Explore/investigation but do not supply the current dashboard panels.

The operational drift-monitor runs every 60 seconds on verification events and feedback. It calculates custom PSI for face_score, voice_score, face_quality, voice_quality and risk_score; Evidently DataDriftPreset selects methods based on the data. Therefore, not every Evidently statistic should be called PSI. Performance reporting uses ClassificationPreset plus FAR/FRR, separating operator human feedback from legacy synthetic-simulation feedback.

View	Content and interpretation
Grafana System Overview	Operational telemetry; investigate freshness/labels when No data appears
Grafana Simulation	Synthetic evidence; snapshots remaining after reset are historical
Reports through the Grafana gateway	data-drift, model-performance, synthetic-performance, data-quality, model-evaluation, pipeline-status, responsible-ai and alerts
Prometheus Query and Targets	Enter a query such as up; an empty query page does not mean data loss
Alertmanager	Group and route firing/resolved alerts
Lifecycle API and MLflow	Actual states and versions; a legacy bundle report alone does not prove a modality rollout

Prometheus rules send alerts to Alertmanager. Production alerts pass through ops-monitor to Telegram when configured. Simulation uses a separate receiver and does not send Telegram notifications. Business check results go to the customer through API/webhooks, not employee-by-employee Telegram messages. Evidently supplies operational reports rather than an additional retraining trigger beside the Airflow decision path.

Employee IDs, raw media and embedding vectors are not Prometheus labels. Denominators, labels and freshness are prerequisites for interpreting performance; a scrape target may be UP while its report still lacks sufficient evidence [S15].

13 CI/CD and Application Deployment

The FSB workflow runs quality on GitHub-hosted Ubuntu, deployment-preflight on Windows, container builds on Ubuntu and deploy-demo on a self-hosted Linux WSL runner. The Windows job runs two exact-SHA staging/mismatch-rejection tests with a temporary runtime and no Docker daemon requirement. Its JUnit results must contain no skips, failures or errors. A green Ubuntu quality job does not establish that these Windows tests ran.

Job	Work performed	Evidence
quality	Ruff, compilation, generated dashboard consistency, pytest/coverage and Compose configuration	quality-evidence with JUnit and coverage
deployment-preflight	Mandatory execution of two Windows tests	deployment-preflight-evidence
containers	Build service images after both preceding jobs	Build logs
deploy-demo	Trusted main; Linux WSL runner calls PowerShell/Docker Desktop	deployment-monitoring-evidence

The deployment runner uses the labels self-hosted, Linux and ddm501-linux-demo. The workflow transfers the exact checkout to Windows, checks the full SHA, stages the release outside OneDrive and preserves runtime secrets, project name, volumes, models, data and reports. This is application replacement on one Docker Compose host, potentially with interruption. It is not application canary deployment across replicas, an ALB or Kubernetes. The project's canary operates at the threshold-policy layer inside the API.

Pull requests do not deploy to the local runner. Pushes changing only Markdown or docs are excluded from automatic workflow execution by path filters. PR/manual dispatch can still run CI, and manual deployment requires main plus the deploy input. The 80% coverage gate applies only to modules declared in the workflow, not the entire UI, DAGs, CLI tools, pretrained weights or biometric quality [S7, S16].

14 Testing and Verification Results

14.1 Test strategy

Tests cover API/tenant behavior, media validation, idempotency, immutable events, outbox, snapshot/evaluation, drift persistence, template poisoning safeguards, retention, lifecycle routing, canary failure and registry integration. MLflow integration tests exercise registration, aliases, reconciliation and rollback against a local registry; data and labels remain synthetic fixtures. SQLite/local tests do not replace load and concurrency testing on production PostgreSQL.

Area	Important behavior	Source
Drift	Input drift does not train; missing labels are not healthy; windows must be distinct	test_drift_decision.py
Serving lifecycle	Shadow retains champion responses; canary/rollback route correctly	test_live_lifecycle.py
Templates	Candidate evidence, holdout and rollback	test_template_lifecycle.py

Area	Important behavior	Source
Registry	Independent aliases, registration and recovery from interrupted promotion	test_lifecycle_registry_integration.py
Monitoring/simulation	Aggregate telemetry, synthetic scenarios and reset	test_lifecycle_monitoring.py, test_simulation.py
Deployment	Exact-SHA staging and mismatch rejection	test_deploy_local.py

14.2 Recorded CI evidence

FSB run **37176107308** for revision **22f64daa7f31181130d5b1355c97eb62eeac5ab6**, recorded on 4 October 2026, reports success for quality, deployment-preflight, containers and deploy-demo. Ubuntu recorded 161 test cases with two skips; Windows executed two tests without skips. Deployment checked 20 queries for the overview dashboard. The run is linked in Appendix A [S4, S16].

The local simulation-monitoring update on the same day recorded 163 passing tests and finite values for the simulation dashboard's 10 panels/17 queries. These results belong to the revision/worktree at that time. They are not combined into the test count of a new CI run or redated to the report-writing date. No full test rerun or redeployment was performed solely to prepare this report.

14.3 Four simulation runs with saved evidence

Modality	Scenario	Outcome	Run ID
Voice	Promotion	SUCCEEDED, reset PASS	06dcd24d-0bfe-4667-9791-924456653862
Face	Rollback	ROLLED_BACK, reset PASS	1e17fc9a-9db7-4fa6-b57a-a6130356a7c4
Face	Promotion	SUCCEEDED, reset PASS	f819f41e-9254-44bf-8e69-d605b2f6e7c6
Voice	Rollback	ROLLED_BACK, reset PASS	1187a0ca-45cb-4ff0-a251-a5aeb088c626

simulation-verification.json records status PASS, synthetic=true and production_policies_unchanged=true. This is a saved comparison for four runs on 4 October, not a claim that runtime state never changes. Local evidence reports are ignored by Git and are not automatically available in a fresh clone. The final report itself is explicitly tracked [S17].

15 Simulation and Demonstrated Results

15.1 Construction and isolation

Simulation receives requests from the platform portal and uses SQLite, file-backed MLflow and a separate volume. It does not receive production database/MinIO credentials or model weights. Airflow, Prometheus, Alertmanager and the host are shared, so CPU/RAM contention remains possible. Synthetic artifacts reside in the simulation volume rather than production MinIO.

The generator creates 201-dimensional unit vectors and synthetic genuine/impostor truth; it does not run raw images/WAV through encoders. It constructs three new drift windows with stable quality, embedding/score

shifts and performance degradation across multiple age cohorts. The baseline threshold is 0.8. Calibration and candidate registration proceed only after the real Alertmanager alert for that run is received. The new threshold is an algorithmic result rather than a hard-coded promotion outcome.

Demo gates are shortened to 200 HTTP requests per stage, a default minimum of five seconds, 20 labels per class and a disagreement allowance of 0.6. Runtime preview may lengthen stages; inspect the run's limits. Production retains 1,000 observations, 3,600 seconds, 30 labels per class and disagreement 0.1. Demo completion time is not an estimate of production rollout duration.

15.2 Face promotion on synthetic data

Face promotion run f819f41e-9254-44bf-8e69-d605b2f6e7c6 produced a candidate threshold of approximately 0.401. The paired offline holdout contains 30 genuine and 45 impostor comparisons: champion FMR 0 and FNMR 1; candidate FMR 0 and FNMR 0. Both have EER 0 because the scores remain separable; the champion's threshold is simply too high for the constructed distribution. This demonstrates threshold adaptation, not improved encoder learning.

Stage	Total HTTP requests	Requests served by challenger
Shadow	200	0
Canary 5%	200	12
Canary 10%	200	20
Canary 25%	200	58
Canary 50%	200	113
Canary 100%	200	200

The saved stages contain no response_mismatches. After promotion, 200 probes used version 2. Routed counts demonstrate actual HTTP policy selection and may differ from configured percentages because of cohort hashing and sample size. Another run need not reproduce exactly 12/20/58/113 challenger requests.

15.3 Voice rollback after increased false matches

Voice rollback run 1187a0ca-45cb-4ff0-a251-a5aeb088c626 passed offline, shadow and initial canary stages, then deliberately injected high impostor scores at 25%. Challenger FMR in the stage evaluation rose to 1 while champion FMR stayed at 0. The gate stopped the challenger; rollback probing recorded 200 samples, served_candidate=0 and version 1 only. ROLLED_BACK means the failure-containment scenario succeeded, not that a new model was promoted.

The baseline is designed to illustrate drift, so the retained champion may still have poor FNMR on shifted samples despite protecting FMR. Rollback reduces the risk introduced by an update; it does not automatically fix the data problem or establish a perfect incumbent. This distinction must be explained during the demonstration.

15.4 Three demo actions and completion criteria

The first button runs drift leading to promotion; the second injects a canary failure leading to rollback; the third cancels/resets simulation routing and aliases to baseline while preserving audit history. Reset does not delete tenants/checks created during the product demo. After each run, export JSON, match run IDs between the portal and Airflow, inspect the simulation model at MLflow :15031 and confirm reset completion.

The defense allocates 15 minutes to presentation, 10-15 minutes to demonstration and 10 minutes to Q&A. If the scheduler is slow or an alert has not arrived, describe the actual waiting state. Saved evidence may be shown with its date/run clearly identified. Do not lower gates, relabel synthetic evidence as human evidence or modify production aliases to make a demonstration appear successful [S17, S18].

16 Responsible AI and Data Protection

16.1 Fairness and evidence quality

The RAI report measures accuracy/FAR/FRR across low, medium and high quality slices. Human slice comparisons require at least 20 labels per slice, including at least five genuine and five impostor labels. An accuracy gap exceeding 10 percentage points is flagged, and Wilson 95% intervals express uncertainty in error rates. Quality slices proxy capture conditions; they do not establish demographic fairness by gender, age or population group.

The DAG generates an RAI report before the offline gate, but the modality lifecycle's candidate endpoint does not currently enforce REQUIRE_HUMAN_FAIRNESS. That flag applies to the legacy bundle gate. Fairness must therefore not be described as an enforced automatic promotion gate in the new flow. Enforcement and an appropriate consented benchmark are needed before a pilot.

16.2 Explainability and human oversight

Responses include scores, thresholds, versions, reasons and margins. The policy explanation path supports sensitivity around the threshold at ± 0.05 and single-modality counterfactuals while other conditions remain fixed. This explains decision rules, not SHAP/LIME or causal behavior inside the encoder. The risk score must not be interpreted as a calibrated probability that an employee cheated.

Human review must separate Face/Voice identity truth from an overall judgment about cheating. Random audits reduce the bias of reviewing suspicious cases only, but do not remove every bias. The service supplies decision support; customers must design recapture, appeal and employee-impacting decision processes. Hosted compatibility includes manual review but not complete customer case-management and appeals.

16.3 Privacy and security

Biometric embeddings are sensitive data, not anonymous data. Consent assertions and tenant scoping provide technical controls; customers must still determine lawful basis, purpose and retention periods. API keys are hashed, webhooks use HMAC, evidence downloads require appropriate authorization, and secrets do not belong in source code, images or reports.

TLS, SSO/RBAC, encryption/KMS policy, access auditing and erasure across PostgreSQL, MinIO and backups need completion before real rollout. Deactivation is not erasure. Research detectors lack sufficient benchmarks for physical replay, unseen deepfakes, accent/disability effects and all user groups. Data must not be repurposed for surveillance or 1:N identification outside the agreed purpose [S6, S9].

17 Operations, Recovery and Scale

17.1 Setup and readiness

Prepare Docker Desktop with Linux containers/WSL2 and Compose v2; use Python 3.11 for host scripts/tests if needed. Clone the FSB repository outside OneDrive, create .env from the example only if it does not exist, set appropriate credentials and run `docker compose up -d --build --wait --wait-timeout 900`. Model-init downloads pinned weights; inspect its logs if the first start is slow. Compose references service-specific Dockerfiles rather than a root-level Dockerfile.

An empty installation does not yet have a one-command procedure to evaluate and initialize a real champion. `/health` may pass while `/ready` returns 503. The handed-over demo runtime must retain or restore an authorized backup of its matching database, artifacts and configuration. Simulation creates its own synthetic baseline and does not require changing the production champion.

Page	Local address
Portal / employee example	http://localhost:18501 / http://localhost:18600
API documentation	http://localhost:18100/docs
Airflow	http://localhost:18081
Production / simulation MLflow	http://localhost:15030 / http://localhost:15031
Grafana overview	http://localhost:13000/d/biometric-overview
Grafana simulation	http://localhost:13000/d/biometric-simulation
Prometheus / Alertmanager	http://localhost:19090 / http://localhost:19093
MinIO console	http://localhost:19101

The private overlay disables legacy simulation endpoints, requires HTTPS callbacks and moves legacy-demo into a separate profile. It does not remove every simulation service/DAG inherited from Compose, so it is not a complete production-hardening configuration [S7].

17.2 Incidents and rollback

Incident	Response	Post-recovery evidence
Canary regression	Stop challenger traffic; retain champion	Failure audit, registry and routed responses
Rollback after promotion	Use the modality lifecycle rollback endpoint	Database champion matches MLflow current/previous versions
Template problem	Operator invokes template rollback	ActiveTemplate pointer and audit
Webhook failure	Preserve outbox and retry; receiver deduplicates	Delivery status and canonical check
Registry/database outage	Restore dependency and reconcile intent	Correct readiness, state and versions; health alone is insufficient
Failed application release	Deploy a schema-compatible previous release	Health, smoke checks and preserved data/volumes

Backups must include PostgreSQL, required MinIO artifacts/evidence, weights and configuration/secrets stored separately. Test restoration into a new environment before relying on the recovery plan. Do not overwrite a serving installation or use `down -v` to reset the demo. Code rollback does not automatically reverse schema/data changes. Automatic application rollback is not implemented in the same way as policy-gate rollback.

17.3 Capacity and cost

Fifty thousand enrolled employees are different from 50,000 concurrent request senders. If N employees are taking an assessment and submit a batch every 30 seconds, the arrival rate is approximately $N/30$ checks per second. At 1,000 concurrent employees this is 33.3 checks/second; at 50,000 it is 1,666.7 checks/second. No load test establishes that the local stack meets either rate.

A preliminary sizing formula is $\text{workers} \geq \text{ceil}(\text{arrival_rate} * \text{CPU_service_time} / 0.65)$, where `CPU_service_time` must be measured for a full check including encoders and inspectors. This is only an estimate. Concurrency locks, database contention, media uploads, model-cache memory, p95 and bursts still require measurement. A single identity-verification timing does not represent full-batch latency.

Monthly cost includes API/worker/control-plane compute, database, storage and requests, network, backups and human review. Discarding raw media by default reduces storage, but evidence, templates and versioned datasets still grow. A 100-sample drift window and 80% identity overlap may not suit a population of 50,000; representative cohorts/windows are needed, not merely more replicas. No measured ROI or cloud bill is claimed [S19].

18 Team Responsibilities and Handover

Team member	Responsibility	Deliverables to verify
Trịnh Đức Dương	Project lead, architecture and core biometrics/lifecycle	Service boundaries, Face/Voice policies, tenant contract and integration review
Đỗ Quang Hiệp	QA, evaluation validation and reproducibility	Regression tests, holdout checks, monitoring evidence and demo validation
Tô Thanh Hải	Platform, containers, CI/CD and operations	Runner/Compose, exact-SHA deployment, recovery and portal integration
Ngô Anh Đức	Reports, slides, diagrams and documentation	Final report, architecture explanation, demo runbook and Q&A

These are the current responsibilities in CONTRIBUTING, not measured effort statistics. Meaningful contribution evidence should include actual work, PRs/reviews, deliverables and the ability to explain or demonstrate them. Git metadata, especially after rewriting history, does not by itself establish who performed the original work. This report does not assign fabricated commit counts to members.

The proposed workflow uses topic-based feat/fix/test/docs branches, specific commit messages, pull requests into main and a reviewer from another area. Relevant CI checks must pass, and deployment occurs only from trusted main. Handover must identify the revision, runtime configuration, secret/backup locations, authorized demo data and evidence for the exact run. API keys, media and database dumps must not be committed to the repository [S20].

19 Limitations, Roadmap and Conclusions

Priority	Work to complete	Acceptance criteria
Before pilot	Consented dataset, trusted labels and customer risk budgets	Reviewed holdout/cohort metrics, class counts and uncertainty
Before pilot	Fairness enforcement in the actual candidate path	Tests show insufficient/rejected evidence blocks rollout
Before pilot	Initial evaluated champion and cross-store retention/erasure	New-install evaluation procedure; evidenced deletion/restore drill
Operational pilot	Capacity, concurrency, TLS/SSO and application recovery	Load tests based on assessment schedules and measured incident recovery
After pilot	Customer training scope and post-activation template monitoring	Enforced consent/isolation; no automatic cross-tenant training
As needed	Encoder fine-tuning and anti-spoof evaluation	Raw datasets, trainer, compute and independent benchmark before serving

Key trade-offs are using pretrained encoders to focus on lifecycle management within available data/compute; using PostgreSQL for 1:1 verification without unnecessary additional infrastructure; and using single-host

Compose for reproducible demonstrations while accepting high-availability and capacity limits. Stateful shadow/canary improves auditability and rollback but requires labels, time and consistency across components.

The project implements batch verification and evidence-based control over threshold-policy updates. Its strengths are clear product, data and operational responsibilities; independent Face/Voice lifecycles; separation of statistical drift from reviewed degradation; and a recovery path when candidates regress. Simulation and CI show that these mechanisms can be tested, but do not replace real-user benchmarks, fairness assessment or production-readiness evidence.

The next step is to turn the existing gates into a pilot process using representative data, clear usage rights and measured operational performance. A champion is a version accepted under available evidence; continued monitoring and preserved history remain necessary after deployment.

Appendix A Evidence Sources

All paths below are relative to the FSB repository root. Sources were checked on 5 October 2026 in a checkout containing local changes. Runtime evidence artifacts under reports are generally ignored by Git; the final report editions are explicitly tracked when committed. Reverification requires the appropriate runtime/data and a new timestamp. The two unrelated project reports supplied only structural inspiration, not algorithms, measurements or results for this project.

ID	Project source	Evidence covered
S1	README.md, PROJECT_REQUIREMENTS.md	Scope, setup and boundaries
S2	ARCHITECTURE.md, docs/ARCHITECTURE_OVERVIEW.md	Components, flows and trade-offs
S3	docs/MODALITY_LIFECYCLE.md, pipeline/lifecycle_config.json	Methods, defaults and gates
S4	docs/EVIDENCE.md	Dated CI/local evidence
S5	SAAS_INTEGRATION.md, api/app/checks.py	API, tenants, idempotency and webhooks
S6	RESPONSIBLE_AI.md, pipeline/responsible_ai_report.py	Fairness, privacy and enforcement gap
S7	DEPLOYMENT.md, OPERATIONS.md, docker-compose.yml	Deployment, readiness and recovery
S8	pipeline/bootstrap_demo.py, pipeline/data_snapshot.py, pipeline/dataset_ledger.py; reports/data-quality.json	Data sources, lineage and snapshot counts
S9	api/app/biometrics.py, api/app/vendor/README.md, pipeline/download_models.py	Encoders, detectors and pinned provenance
S10	pipeline/evaluation.py, pipeline/modality_training.py	Identity splits, calibration and artifacts
S11	airflow/dags/	Three DAGs and seven model tasks
S12	pipeline/drift_decision.py, api/app/lifecycle_service.py	Drift, persistence and triggers
S13	api/app/template_lifecycle.py	Template safeguards and rollback

ID	Project source	Evidence covered
S14	api/app/model_lifecycle.py, api/app/observation.py, api/app/lifecycle_api.py	Policy routing and transitions
S15	MONITORING_MAPPING.md, monitoring/ and dashboard builders	Metrics, reports and alerting
S16	.github/workflows/ci.yml, tests/test_deploy_local.py	CI jobs and preflight
S17	docs/SIMULATION.md; reports/simulation-verification.json, reports/simulation-promotion-face.json, reports/simulation-rollback-voice.json	Run IDs, offline results, routed counts and probes
S18	docs/presentation/DEMO_RUNBOOK.md, docs/presentation/QA_GUIDE.md	Live demonstration and troubleshooting
S19	SCALABILITY_COST.md	Capacity/cost assumptions
S20	CONTRIBUTING.md, RUBRIC_MAPPING.md	Team process and assignment mapping

Recorded GitHub Actions run:

<https://github.com/FSB-MSA36HN/DDM501-face-voice-proctoring/actions/runs/37176107308>

Pretrained models and datasets: OpenCV Zoo for YuNet/SFace; SpeechBrain spkrec-ecapa-voxceleb; LFW at marcelohaps/lfw; Speech Commands at mteb/speech-commands-mini; AASIST from clovaai/aasist; MiniFASNetV2 according to attribution and revision information in the vendor README. These models and datasets belong to their respective upstream projects; the team does not claim to have invented or trained these encoders from scratch.

Appendix B Glossary

Term	Meaning in this report
Enrollment / template	Registration / reference vectors associated with a person
Embedding	Vector representation produced by an image or voice encoder
Genuine / impostor	Same identity / different identity according to independent labels
FMR / FNMR	Impostor false acceptance / genuine false rejection rate
PSI / MMD ²	Scalar distribution comparison / kernel distance between vector distributions
Candidate / challenger	Newly registered policy / policy eligible for controlled evaluation
Champion	Accepted policy serving under the lifecycle
Shadow / canary	Observation without changing real responses / routing a cohort to the new policy
DAG	Directed acyclic graph of tasks orchestrated by Airflow
Lineage / fingerprint	Traceable relationships between data and models / content-identifying hash
RAI	Responsible AI, including fairness, explainability, privacy and human oversight
Synthetic evidence	Evidence generated from constructed data, not a real-user benchmark